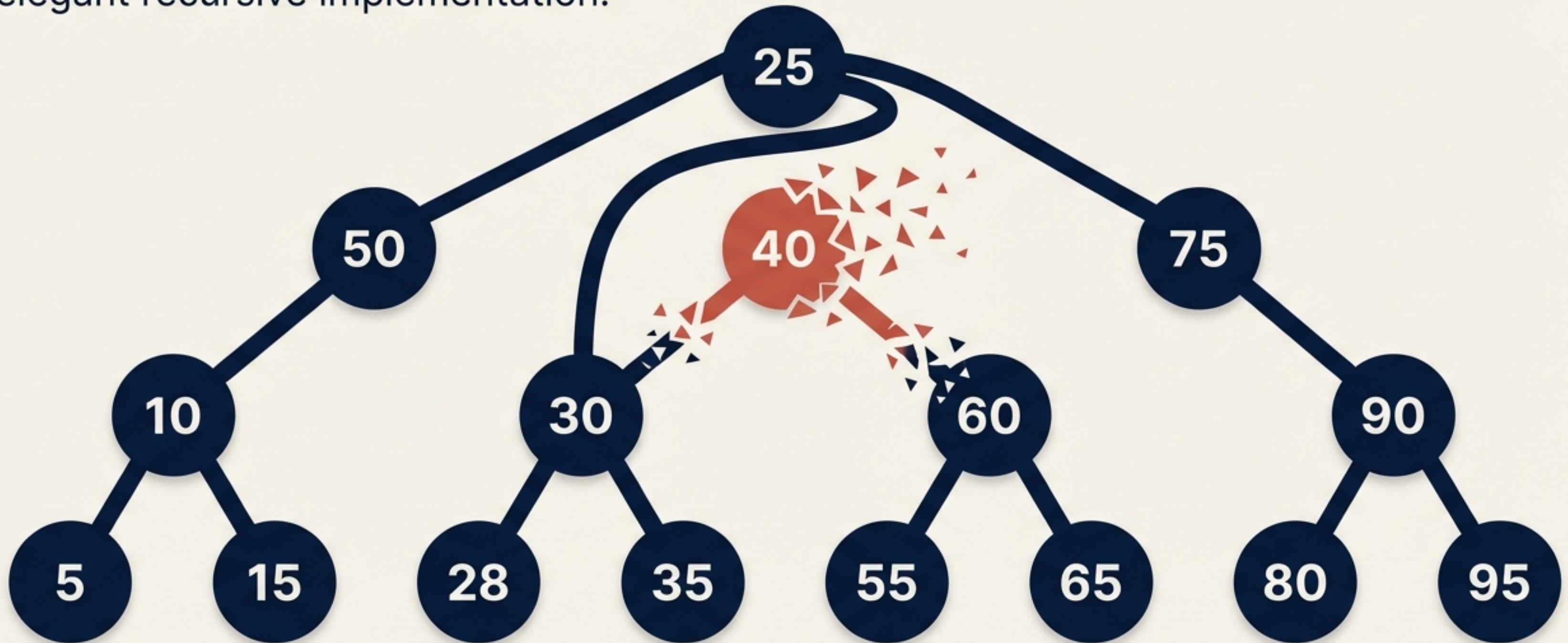


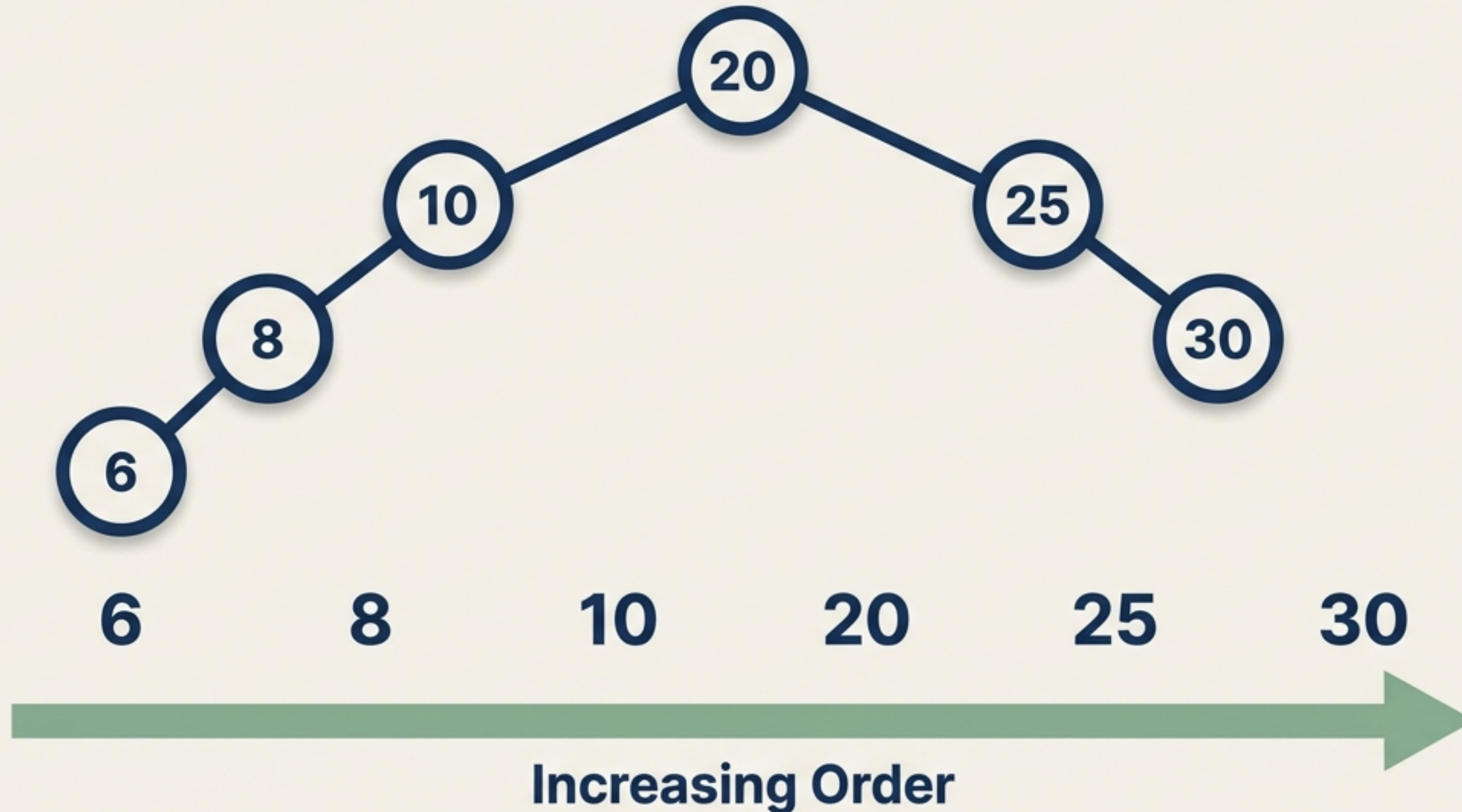
Deletion in Binary Search Trees

A visual guide to algorithmic logic and elegant recursive implementation.



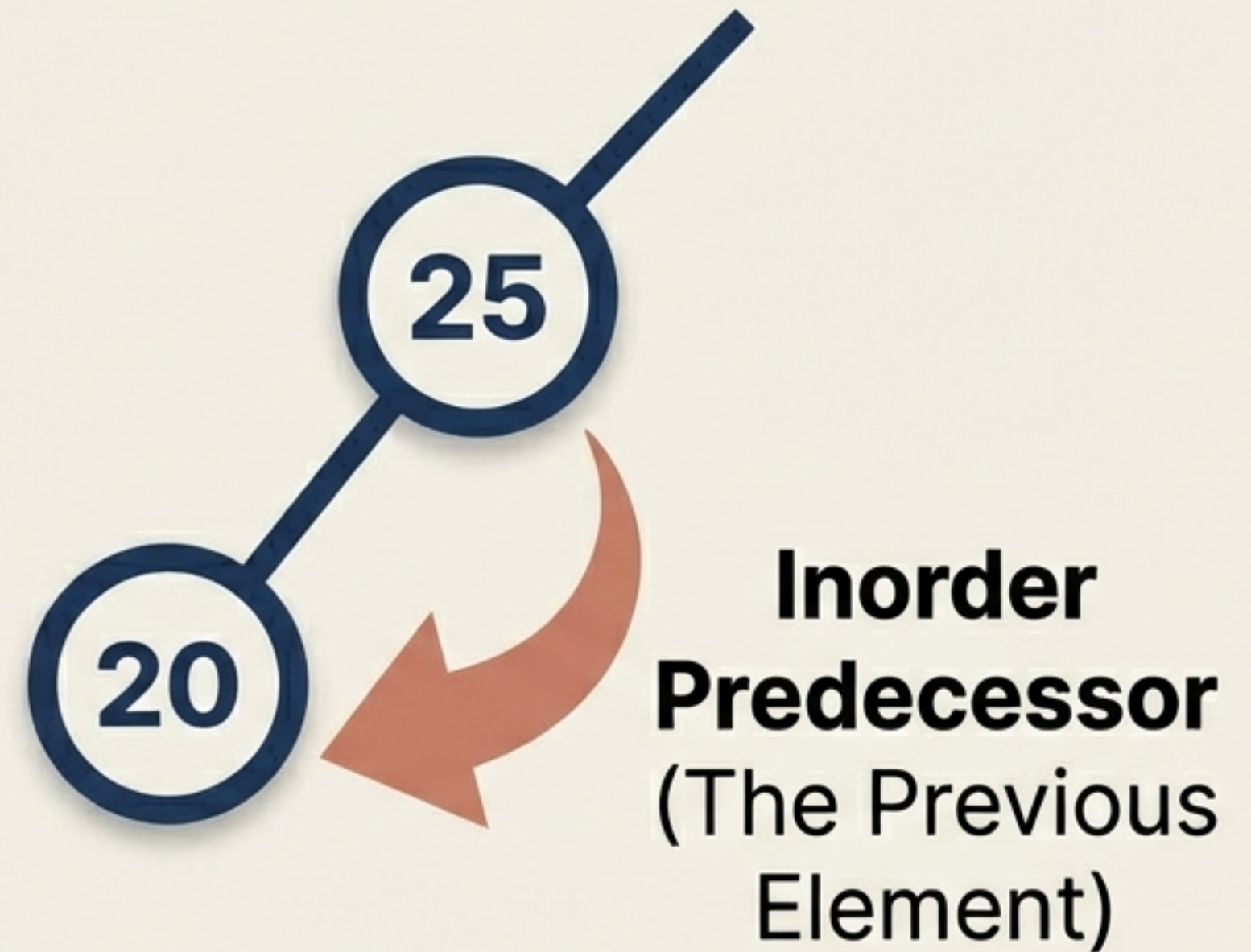
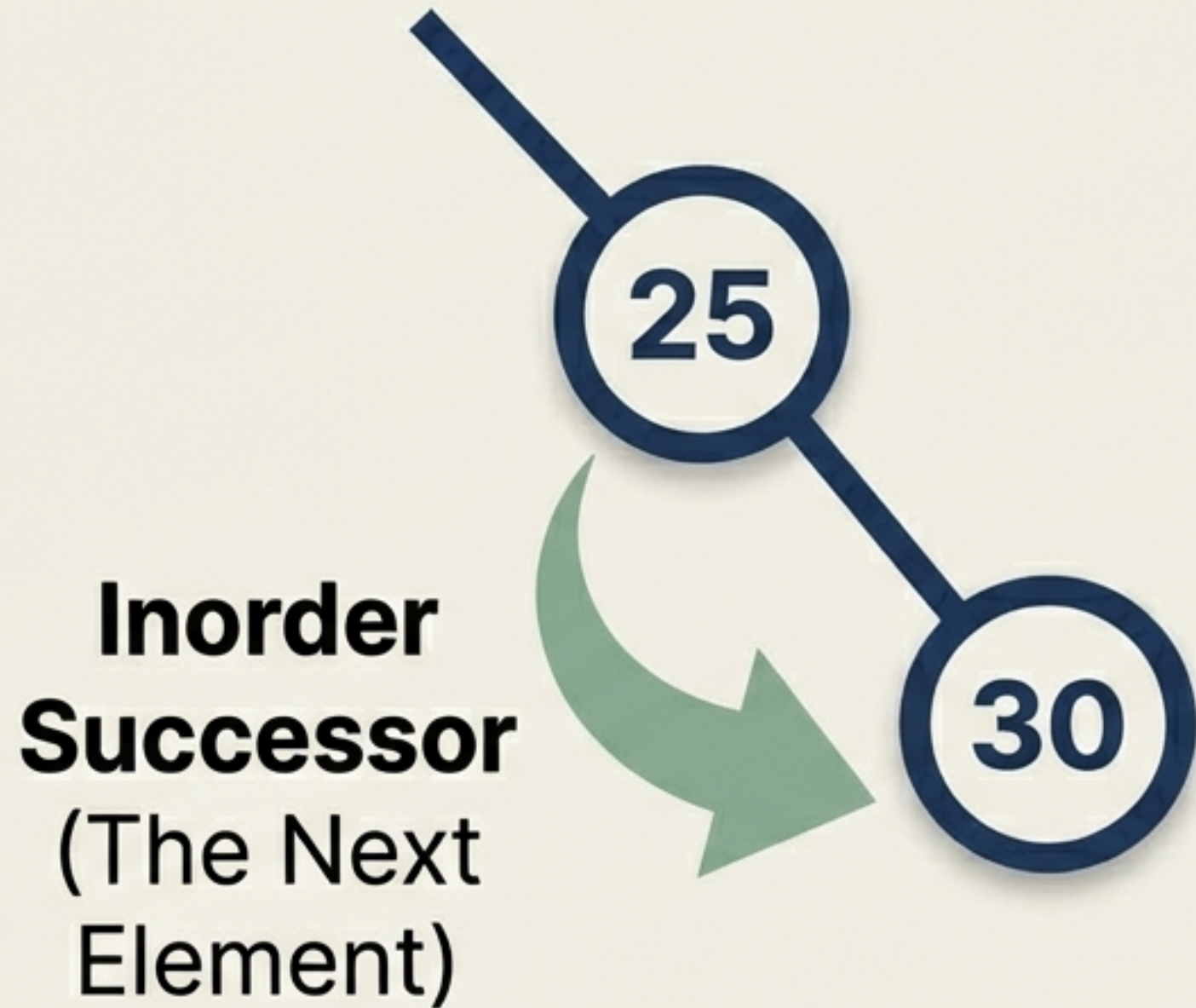
The Golden Rule of the BST

- The Constraint: An Inorder Traversal of a BST must always print elements in strictly increasing order.
- The Goal: We must be able to remove any node without breaking this hierarchical rule.
- Jumping straight to code leads to memorization. Understanding the visual logic leads to mastery.

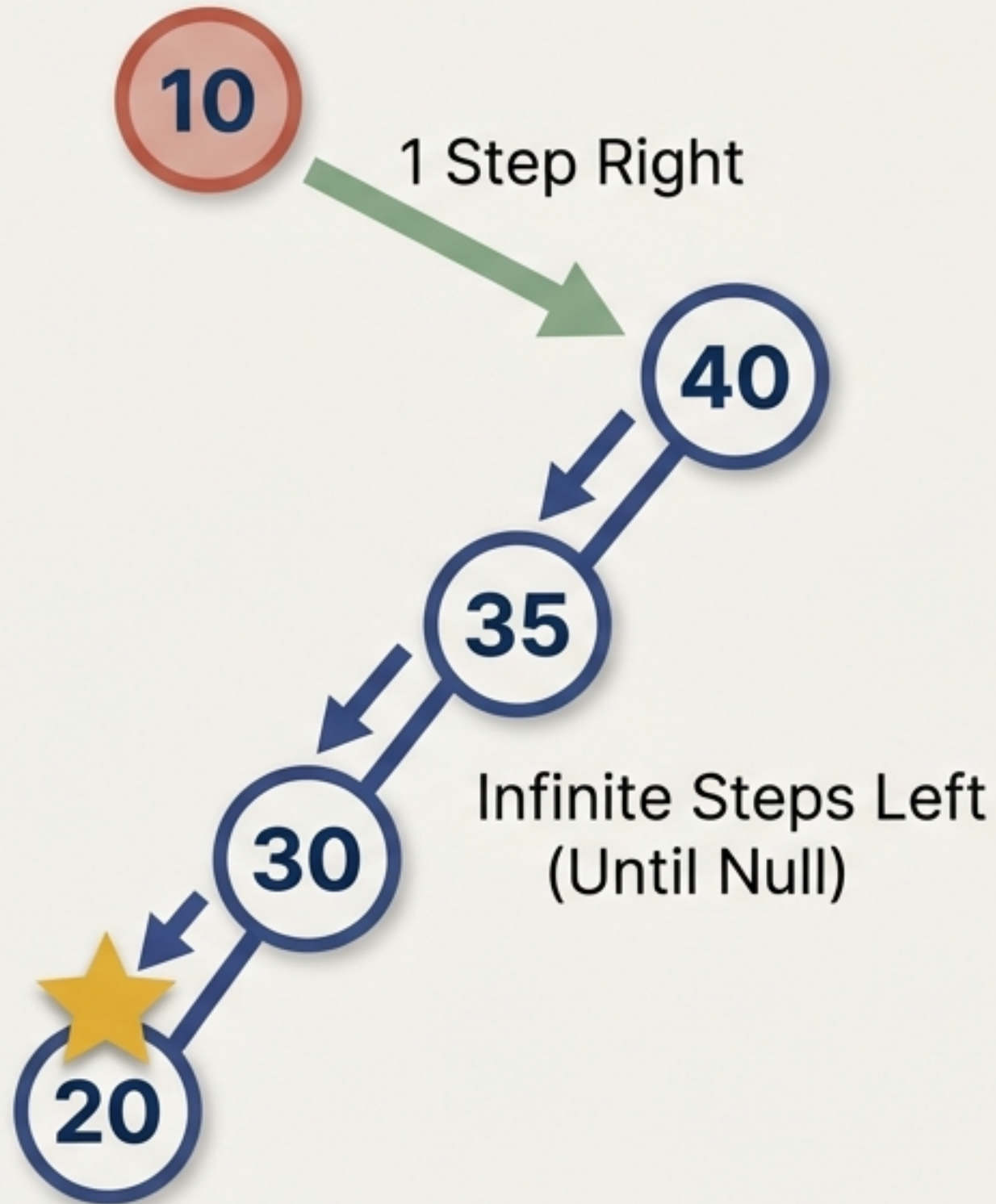


The Tools: Successors and Predecessors

Before deleting nodes, we must know how to find a node's immediate numerical neighbors within the tree structure.



Finding the Successor Map



The Successor Algorithm: Go one step towards the right child, then travel down the left-most path until you cannot go any further.

The Three Scenarios of Deletion

Every deletion in a BST falls into one of three structural cases based on the target node's dependencies. We scale our logic from the simplest to the most complex.

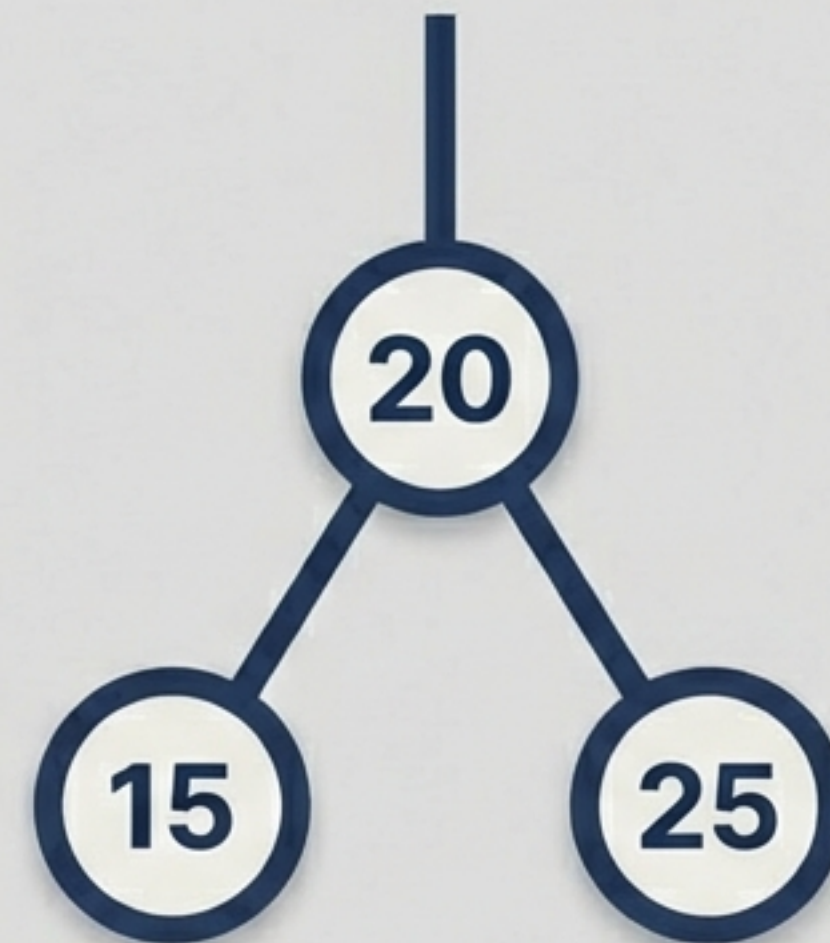
0 Children
(Leaf Node)



1 Child



2 Children



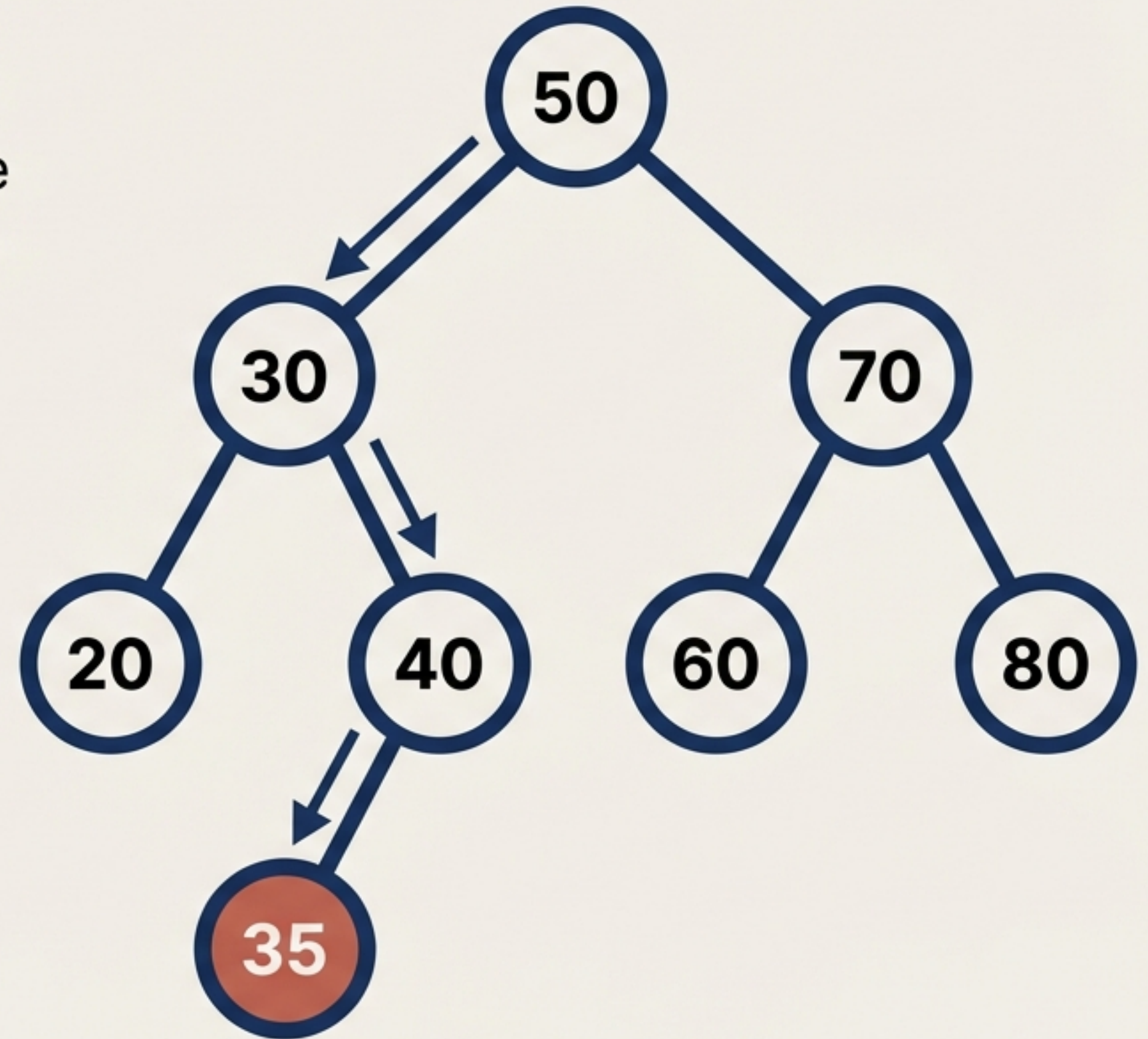
Phase 1: Locating the Target

Deletion requires traversal. Like insertion or searching, we must first recursively navigate the tree to find the precise memory address of our target node.

If Target < Current Node
→ Search Left Subtree

If Target > Current Node
→ Search Right Subtree

**If Target == Current Node
→ Execute Deletion**

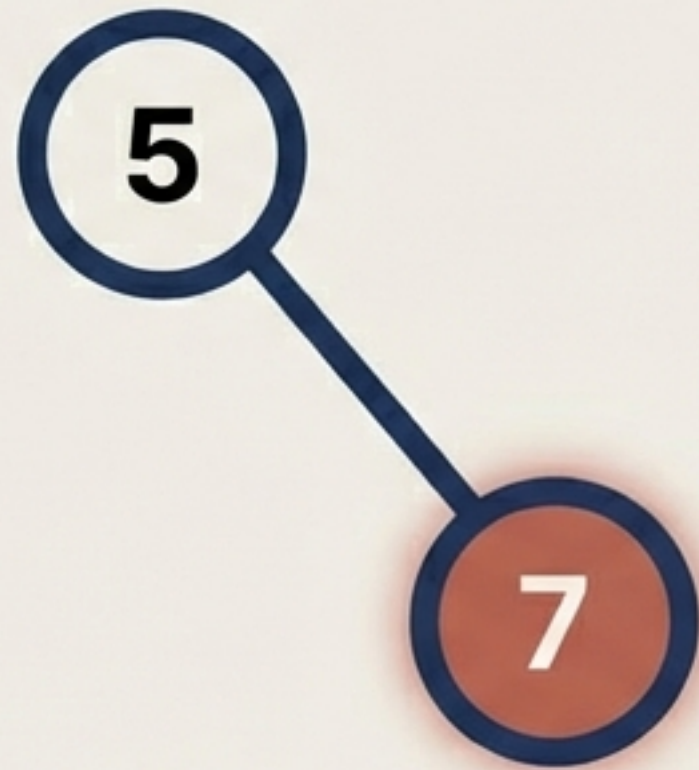


Case 1: The Leaf Node (0 Children)

The Logic: The target node has no dependencies.

The Action: Simply sever the tie. The recursive function returns Null to the target's parent, completely disconnecting the node from the tree.

Before



After



Case 2: One Child

One Child

The Metaphor: Handing over responsibility.

The Logic: If the target node is deleted, who takes care of its only child? The target's parent.

The Action: Return the address of the target's single child up to the target's parent.

Before

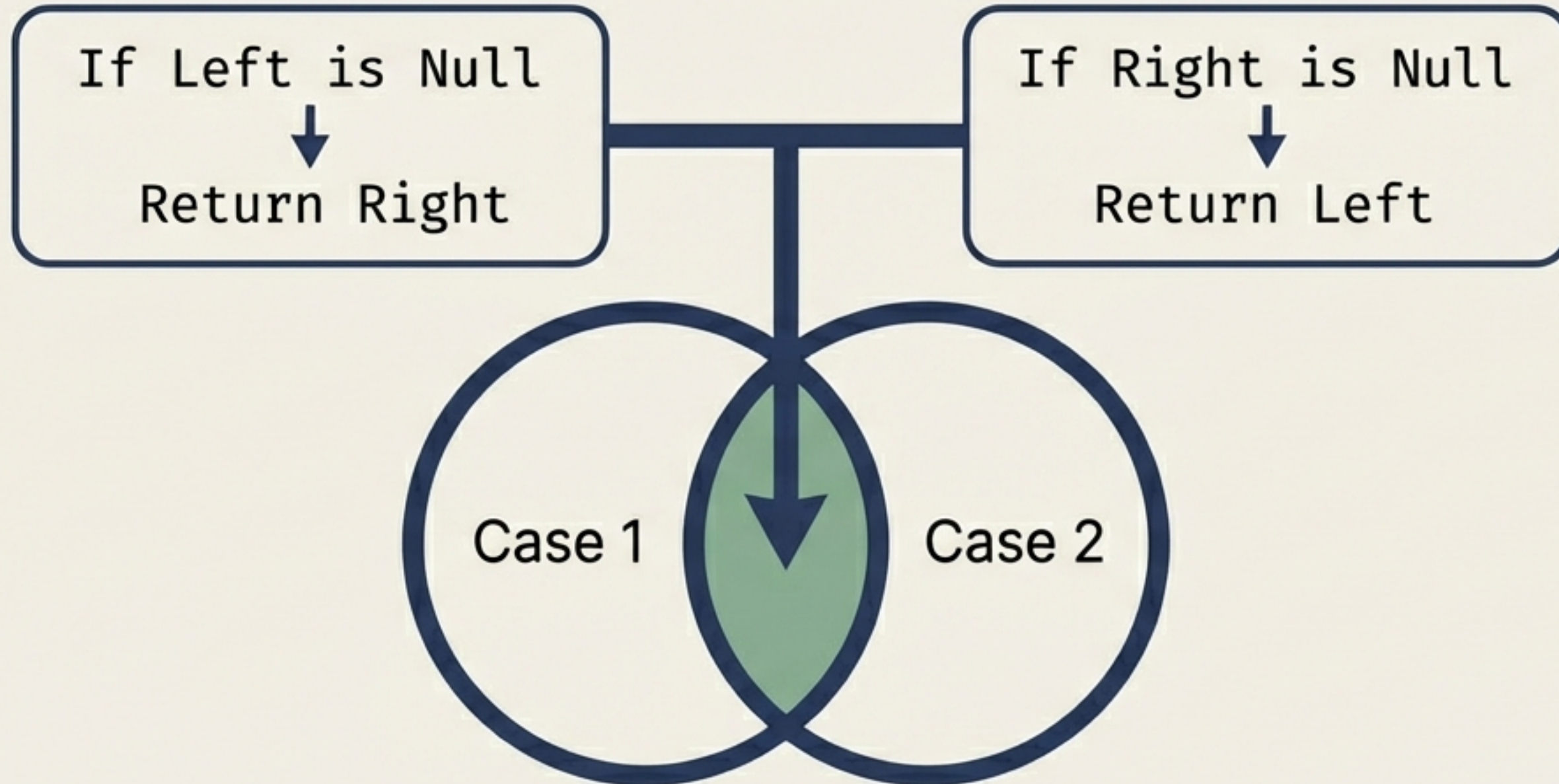


After



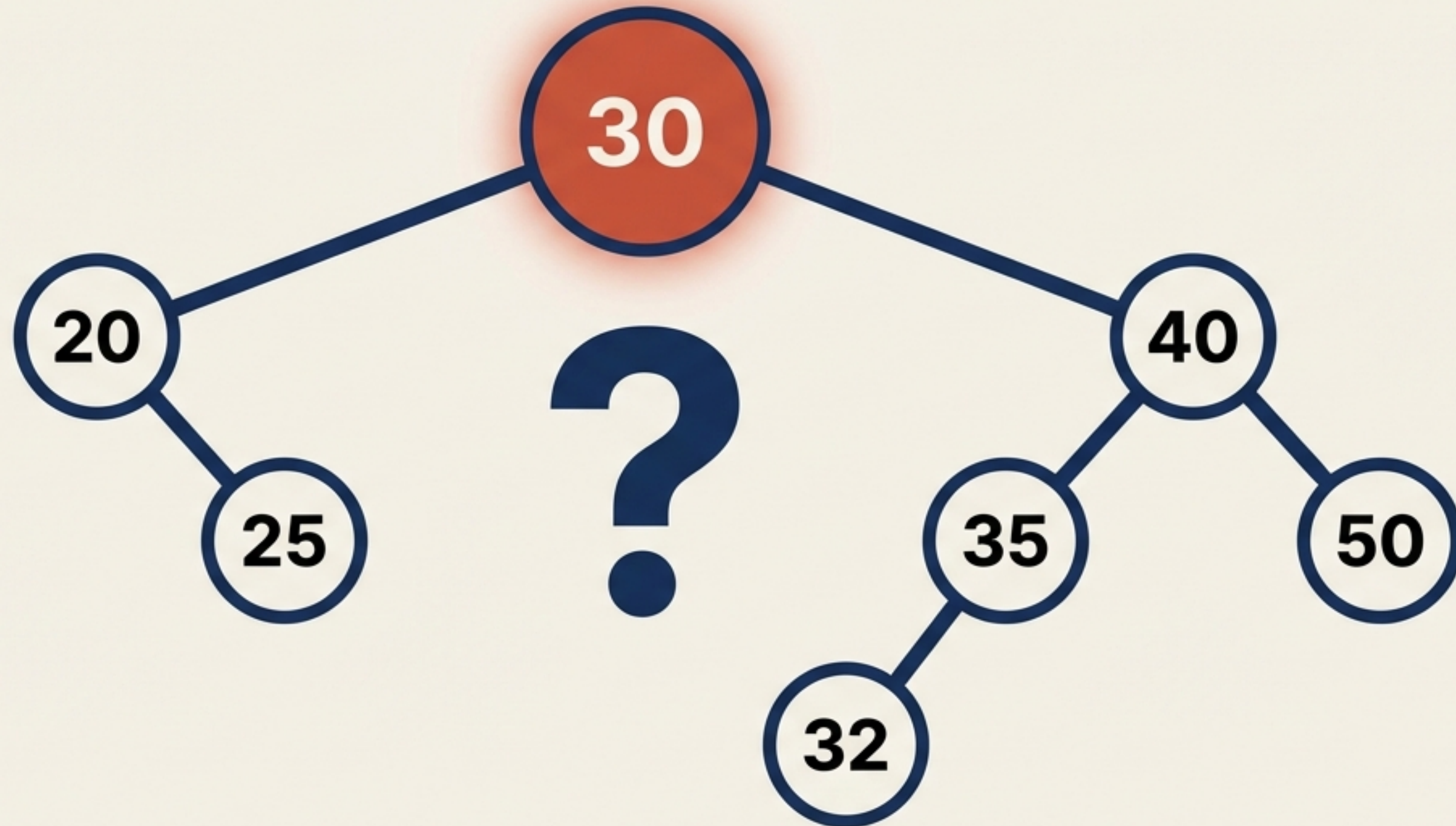
The Elegant Shortcut: Uniting Cases 1 & 2

We never explicitly code Case 1. If a node has no left child, we return its right child. If it's Case 2, it returns the existing right child. If it's Case 1 (no right child either), it naturally returns Null. Both cases are solved instantly.



Case 3: Two Children (The Boss Level)

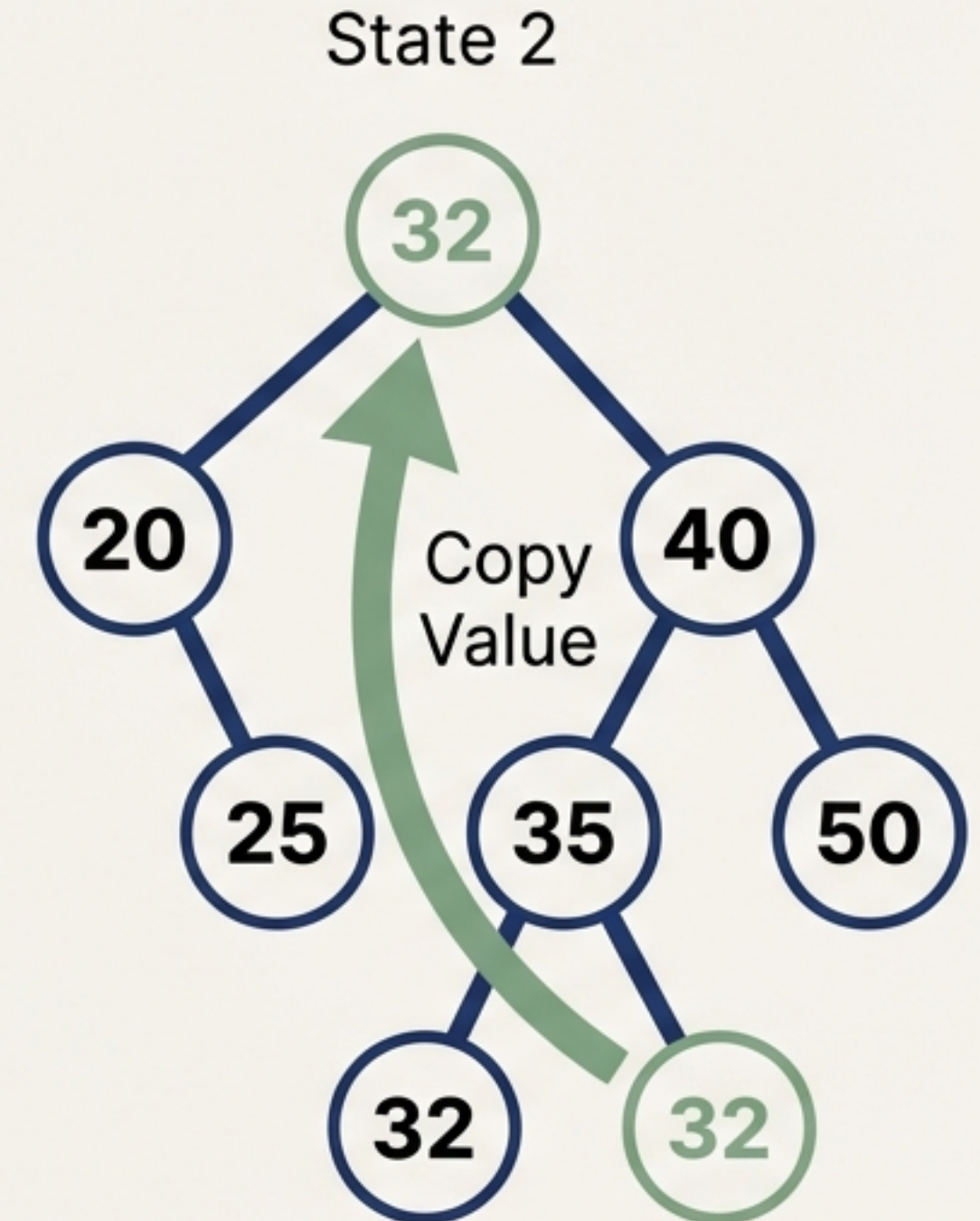
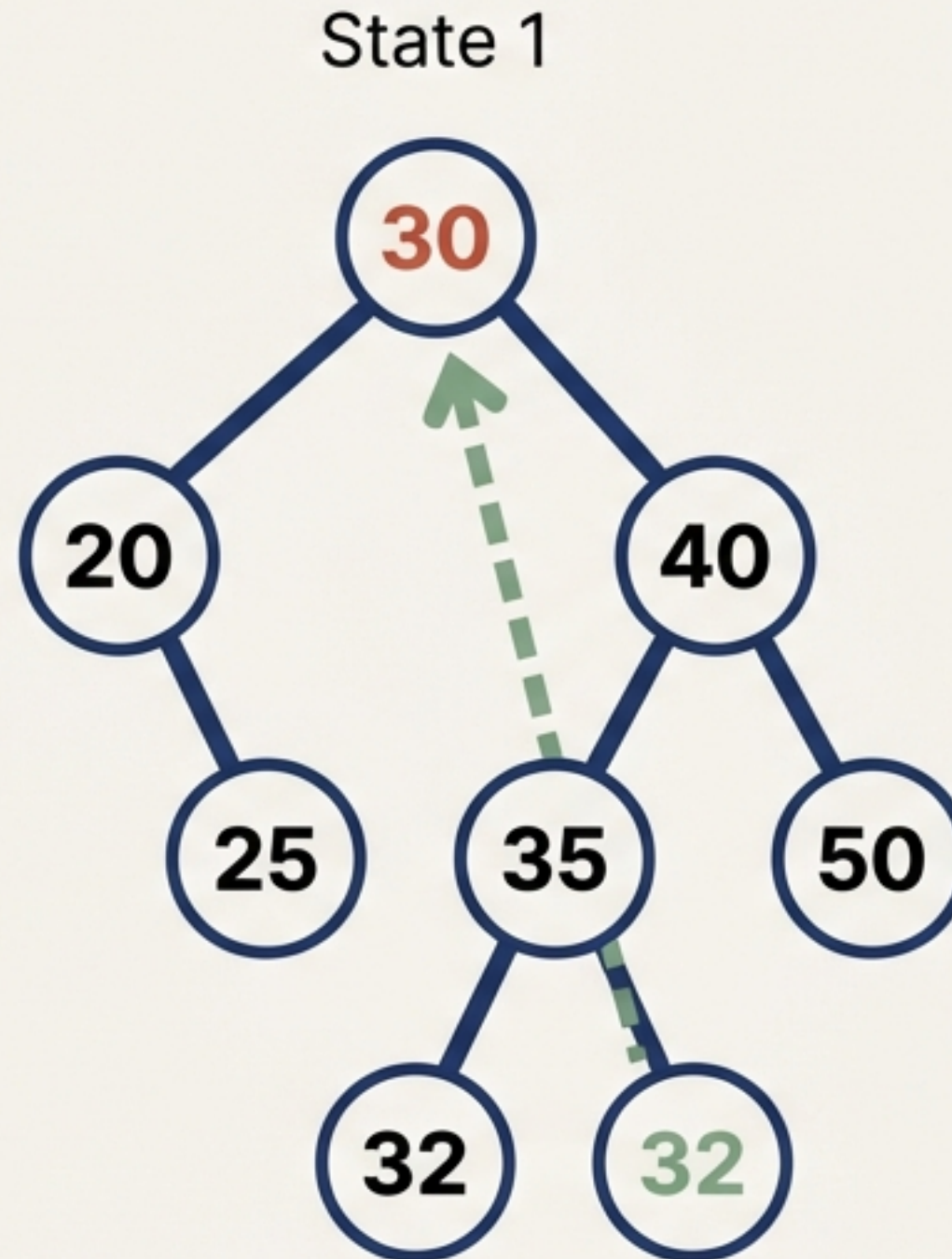
If we delete a node with two children, the parent cannot adopt both subtrees without violating the binary structure (max 2 children per node). We cannot simply hand over responsibility.



The Case 3 Strategy: Substitution

Instead of deleting the structural node, we overwrite its data.

We find the Inorder Successor ('32') and copy its value into the target node. The strict BST sorting rules remain intact.

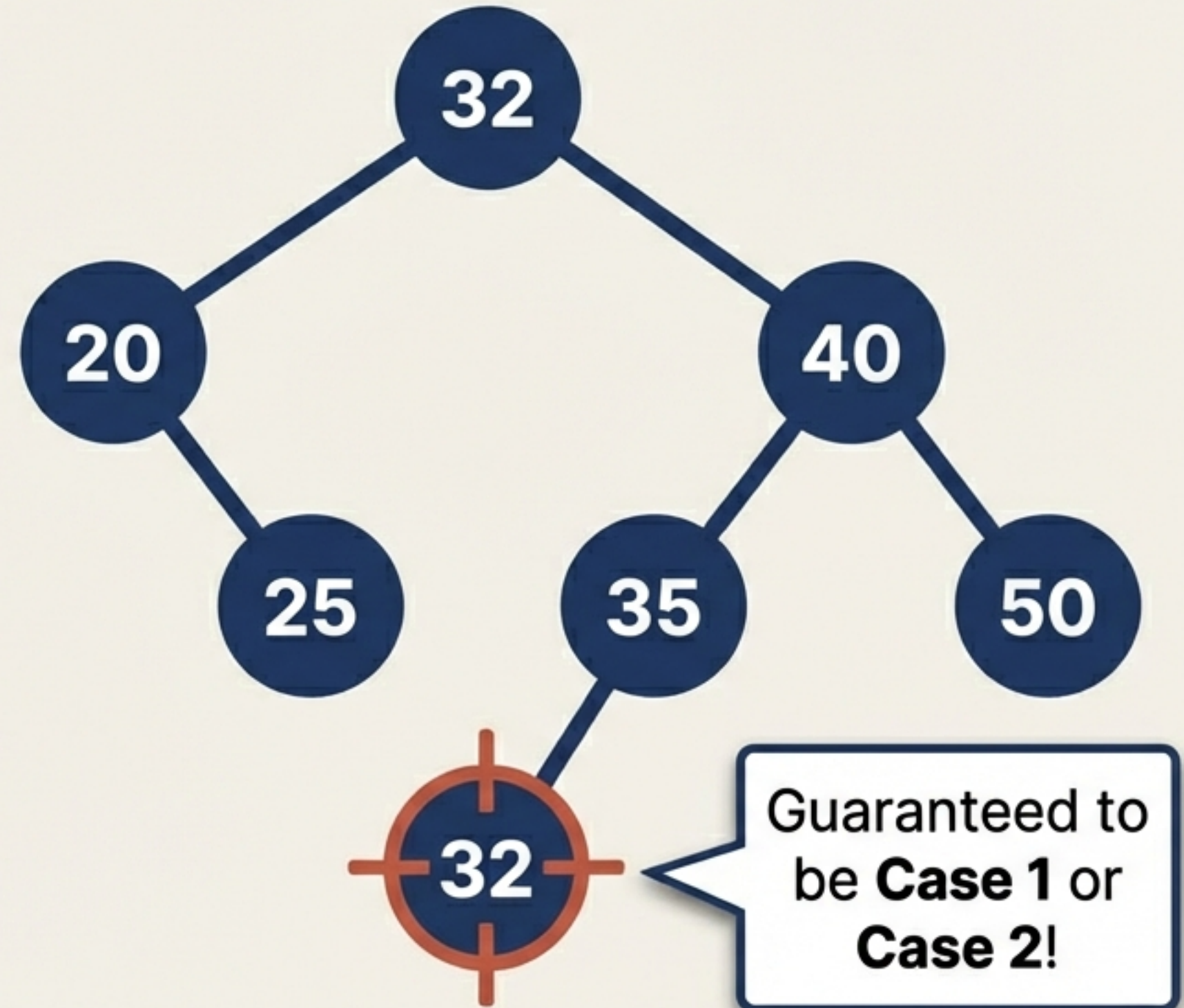


The Case 3 Strategy: Recursive Cleanup

Now, we have a duplicate value.
We must delete the original
Inorder Successor.

The Payoff: Because of how a
Successor is found (traveling left
until you hit a dead end), the
Successor node will never have
a left child.

Therefore, deleting the
Successor is always a simple
Case 1 or Case 2 deletion!



The Case 3 Execution Flow

By passing the deletion responsibility to the right subtree, we elegantly transform our complex Case 3 problem into an already-solved Case 1/2 problem.

Recursion automatically handles the rest via Case 1 or 2

Node has 2 children

Find Successor
(1 Right, Infinite Left)

CurrentNode.Data =
Successor.Data

Delete(CurrentNode.Right,
Successor.Data)

The Complete Deletion Engine

A unified, recursive system. Four distinct steps perfectly balancing data structure constraints with code efficiency. Ensure elif (else-if) logic is used in implementation to prevent cascading conditional errors.

